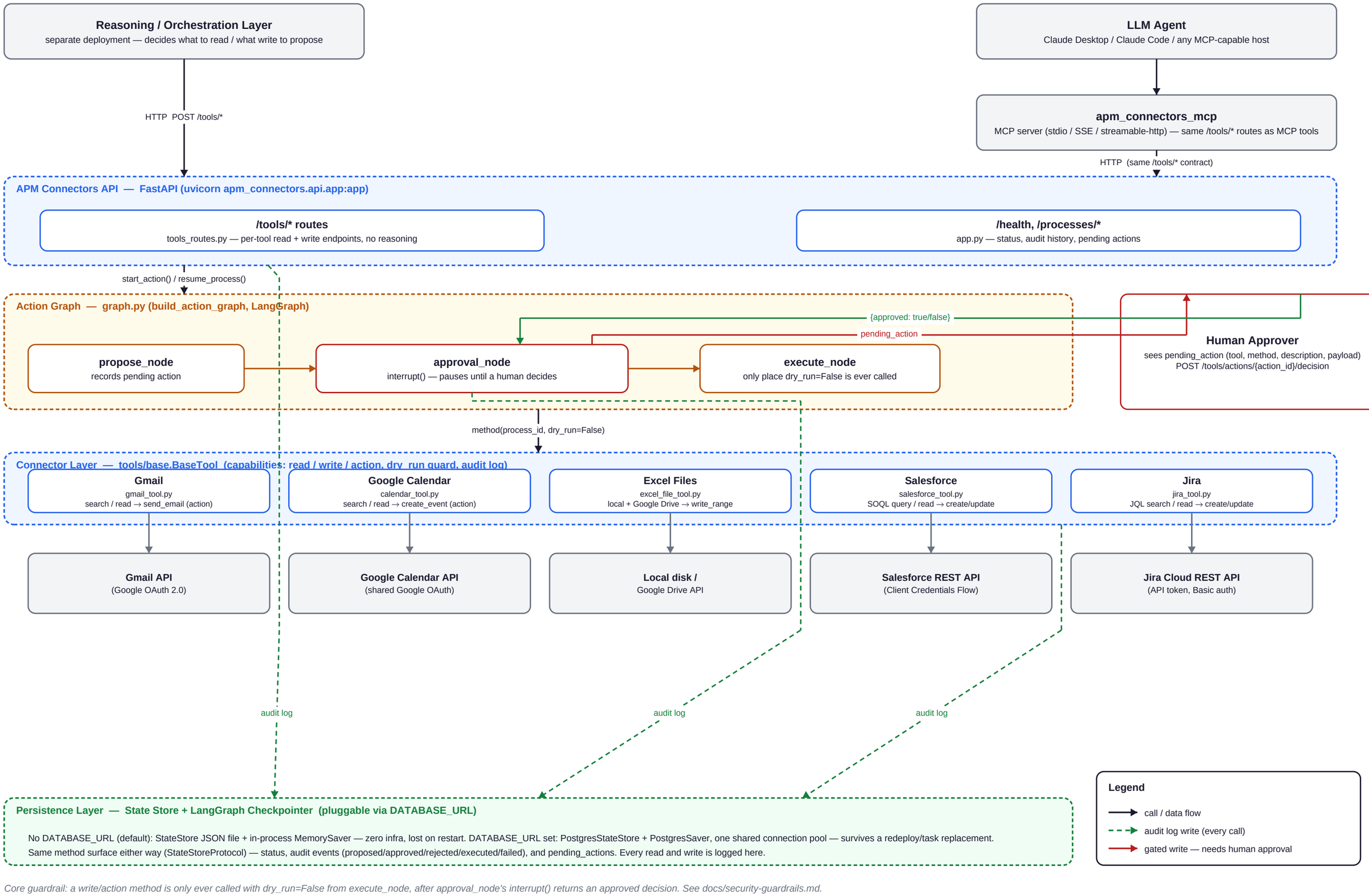
APM Connectors & Enterprise Systems — Architecture

Connector / enterprise-systems layer only — no reasoning, no LLM call, no free-text entry point in this process (docs/api-contract.md)

Updated: 5 live connectors (MS Graph Excel removed), Postgres-backed state store + LangGraph checkpointer (DATABASE_URL), MCP server layer



Core guardrail: a write/action method is only ever called with dry_run=False from execute_node, after approval_node's interrupt() returns an approved decision. See docs/security-guardrails.md.

Component Reference

Every component from page 1, its path, and what it's responsible for -- plus the connector and persistence details that don't fit on the diagram.

Components

Component	Path	Responsibility
Tool connectors	src/apm_connectors/tools/*.py	One module per external system. Each implements the common BaseTool interface (tools/base.py): declares capabilities (read/write/action), and every write/action method takes dry_run and logs to the state store.
Action graph	src/apm_connectors/graph.py	A 3-node LangGraph (propose -> approval -> execute) that gates every write. approval_node's interrupt() is the actual mechanism -- the graph physically cannot proceed past it without a Command(resume=...) carrying a human decision.
State store	src/apm_connectors/state/store.py, state/postgres_store.py	Process status, the audit log, and pending (proposed-but-undecided) actions. Two interchangeable implementations behind one method surface (StateStoreProtocol) -- see the persistence table below.
HTTP API	src/apm_connectors/api/	app.py (FastAPI app + /health, /processes/*), tools_routes.py (one read/write route pair per connector), dependencies.py (builds the shared tools, state store, and compiled action graph once per process), schemas.py.
MCP server	src/apm_connectors_mcp/	Exposes every /tools/* route as an MCP tool for an LLM agent host (Claude Desktop/Code). Talks to the HTTP API purely over HTTP (client.py) -- never imports apm_connectors.tools/graph directly, so it deploys independently.
Config	src/apm_connectors/config.py	Reads environment variables into a Settings dataclass, including DATABASE_URL. No secrets ever live in this file -- see .env.example.
Deployment	infra/aws/ecs-fargate/ (Terraform)	Builds/pushes the image, stands up an ECS Fargate service behind a public ALB, stores OAuth/API secrets and DATABASE_URL as SSM SecureStrings. See docs/deployment.md.

Connectors (5 live -- MS Graph/OneDrive Excel connector was removed)

Connector	Module	Read capability	Write/action capability	Auth
Gmail	gmail_tool.py	search, read	send_email (action)	Google OAuth 2.0 -- shared consent with Calendar
Google Calendar	calendar_tool.py	search, read	create_event (action)	shared Google OAuth (same token as Gmail)
Excel Files	excel_file_tool.py	read	write_range	Local disk, or Google Drive API
Salesforce	salesforce_tool.py	SOQL query, read	create, update	OAuth 2.0 Client Credentials Flow
Jira	jira_tool.py	JQL search, read	create, update	API token, Basic auth (jira_email + jira_api_token)

Persistence -- pluggable behind one setting (DATABASE_URL)

	No DATABASE_URL (default)	DATABASE_URL set
State store	StateStore -- local JSON file (state/store.py)	PostgresStateStore -- Postgres tables, same method surface (state/postgres_store.py)
LangGraph checkpointer	MemorySaver -- in-process memory only	PostgresSaver -- same Postgres database, same connection pool
Survives a task replacement?	No -- both vanish on restart, including anything mid-approval	Yes -- live-verified: propose -> replace the ECS task -> pending action + audit trail intact -> approve -> executes
Extra dependency	none	postgres extra (psycopg + psycopg_pool) -- installed unconditionally in the Docker image
Connection resilience	n/a	check=ConnectionPool.check_connection replaces a dead pooled connection transparently (e.g. a serverless Postgres like Neon suspending idle compute)

Key invariants worth remembering

- No write executes without an approved interrupt() resume -- enforced twice: graph.py's execute_node is the only place any tool's write/action method is ever called with dry_run=False, and every connector's own require_dry_run_guard refuses to treat a call as real without that flag. No config flag or "autonomous mode" bypasses this.
- This package has no reasoning of its own -- deciding what to read or write belongs to a separate reasoning/orchestration layer calling this API, not here.
- Adding a tool or a route is additive; changing an existing route's request/response shape is a breaking change for whatever's already coded against docs/api-contract.md.
- Every implementation swap (state store, checkpointer) happens behind an existing protocol/method surface, not by changing what callers depend on -- StateStoreProtocol is the example so far.

Deep Dive — Request Flow, Connectors, Persistence, MCP, Deployment

The mechanics behind pages 1-2's diagram and tables, in reading order: how a call actually moves through the system, the connector interface every tool implements, the pluggable persistence layer behind the approval gate, the MCP front door, and how it all deploys.

1. How a call moves through the system

Every /tools/* route follows one of exactly two shapes, and nothing in this codebase deviates from them. **A read executes immediately** -- no approval, no graph involved:

```
@router.post("/salesforce/query")
def salesforce_query(body: SalesforceQueryRequest, tools=Depends(get_tools)) -> list[dict]:
    tool = _tool(tools, "salesforce")
    results = tool.query_records(process_id, soql=body.soql)
    return [r.__dict__ for r in results]
```

A write never executes on the call that proposes it. The route never calls the tool's write method directly -- it calls start_action, which hands the request to the action graph:

```
@router.post("/salesforce/create", response_model=RunOutcomeResponse)
def salesforce_create(body: SalesforceCreateRequest, tools=Depends(get_tools),
    graph=Depends(get_action_graph)):
    _tool(tools, "salesforce")  # fail fast, before recording a pending action doomed to fail
    description = f"Create Salesforce {body.object_name} record"
    payload = {"object_name": body.object_name, "fields": body.fields}
    return _propose(graph, action_id, "salesforce", "create_record", description, payload)

@router.post("/actions/{action_id}/decision", response_model=RunOutcomeResponse)
def decide_action(action_id: str, body: DecisionRequest, graph=Depends(get_action_graph)):
    outcome = resume_process(graph, action_id, approved=body.approved)
    return to_response(outcome)
```

Every one of the five connectors' write/action routes (gmail_send, calendar_create_event, excel_write, salesforce_create/update, jira_create/update) is the same three lines: look up the tool to fail fast on an unconfigured connector, build a description + payload, call _propose. There is exactly **one** decision route -- /tools/actions/{action_id}/decision -- shared by all of them, because approving or rejecting is the same operation regardless of which connector proposed the action.

2. The connector layer — one interface, five connectors

Every connector (gmail_tool.py, calendar_tool.py, excel_file_tool.py, salesforce_tool.py, jira_tool.py) implements the same abstract base:

```
class Capability(str, Enum):
    READ = "read"
    WRITE = "write"
    ACTION = "action"

@dataclass(frozen=True)
class ActionResult:
    executed: bool
    description: str
    details: dict[str, Any]

class BaseTool(ABC):
    name: str
    capabilities: frozenset[Capability]

    def require_dry_run_guard(self, dry_run: bool, process_id: str, summary: str) -> None:
        """Defense in depth alongside the graph's interrupt() gate -- logs
        action_executed for a real call, action_proposed for a dry run."""
        event_type = "action_executed" if not dry_run else "action_proposed"
        self._log(process_id, event_type, summary, {"dry_run": dry_run})
```

Two things worth noticing. First, **require_dry_run_guard is not the approval gate** -- the graph's interrupt() (section 3) is what actually prevents a write from running. This guard is a second, independent check inside the connector itself: even if something upstream of the connector were ever wired incorrectly, no write method can silently skip logging what it did. Second, **ActionResult.executed** is the one field every caller checks to tell a real write from a dry run or a rejection -- the shape is identical either way, so nothing downstream needs a special case for "this didn't actually happen."

The MS Graph/OneDrive Excel connector that originally existed alongside excel_file_tool.py has been removed entirely (code, tests, docs, Terraform) in favor of the local-file/Google-Drive Excel connector alone — one Excel implementation, not two competing ones.

3. The approval gate — propose, interrupt, resume

graph.py compiles a 3-node LangGraph: propose -> approval -> execute. The middle node is the entire guarantee this package exists to provide:

```
def approval_node(state: GraphState) -> dict[str, Any]:
    decision = interrupt({
        "type": "approval_request", "action_id": action_id, "tool": proposed["tool"],
        "method": proposed["method"], "description": proposed["description"],
        "payload": proposed["payload"], "category": state.get("category", "other"),
    })
    approved = bool(decision.get("approved")) if isinstance(decision, dict) else bool(decision)
    state_store.resolve_pending_action(action_id, approved=approved)
    return {"decision": approved}
```

interrupt() physically stops execution at that line -- the graph cannot proceed to execute_node without a Command(resume=...) carrying a human decision, delivered via the shared decision route. On resume, LangGraph *replays* approval_node from the top; interrupt() is deliberately the first side-effecting statement in the function so replay is harmless, and resolve_pending_action runs exactly once, on the resume pass, after interrupt() returns. execute_node is the only place in the entire codebase where a tool's write/action method is ever called with dry_run=False. Rejecting instead of approving skips that call entirely -- nothing is ever sent, created, or written on a rejection.

4. Persistence — the pluggable state store and checkpointer

Two different things need to survive between a write being proposed and a human deciding on it, and until recently neither one reliably did on this deployment's actual infrastructure (AWS ECS Fargate tasks have no persistent disk, and a redeploy replaces the running task entirely):

	No DATABASE_URL (default)	DATABASE_URL set
State store (status, audit log, pending actions)	StateStore — a local JSON file (state/store.py)	PostgresStateStore — Postgres tables, identical method surface (state/postgres_store.py)
LangGraph checkpointer (the graph's own paused execution state)	MemorySaver — in-process memory only, gone the instant the process dies	PostgresSaver (langgraph-checkpoint-postgres) — same Postgres database

Both are picked by the exact same signal, in api/dependencies.py, and share one connection pool rather than opening two separate sets of connections to the same database:

```
@lru_cache
def get_state_store() -> StateStoreProtocol:
    settings = load_settings()
    if settings.database_url:
        from apm_connectors.state.postgres_store import PostgresStateStore
        return PostgresStateStore(_get_postgres_pool())
    return StateStore()

@lru_cache
def get_action_graph():
    if settings.database_url:
        from langgraph.checkpoint.postgres import PostgresSaver
        checkpointer = PostgresSaver(_get_postgres_pool())
        checkpointer.setup()
    else:
        checkpointer = MemorySaver()
    return build_action_graph(get_tools(), get_state_store(), checkpointer=checkpointer)
```

Callers (tools, routes, tests) only ever depend on **StateStoreProtocol**'s method surface, never on which class is behind it — that's what made this swap possible without touching anything outside state/ and api/dependencies.py. One more detail worth knowing: the pool is opened with check=ConnectionPool.check_connection, which verifies a connection is actually alive before handing it out and transparently replaces it if not. Without this, a serverless Postgres provider (Neon, etc.) suspending its compute while a pooled connection sits idle breaks every call using it indefinitely with "SSL connection has been closed unexpectedly" — live-verified against Neon, and the fix that made it reliable.

> *Live-verified end to end: propose a write → replace the ECS task entirely → the pending action and its audit trail are still there on the brand-new task → approve → it executes. That's the actual scenario a real deployment needs to survive, not just "the server didn't crash."*

5. The MCP server — the natural-language front door

src/apm_connectors_mcp/ is a separate top-level package (its own pyproject.toml extra, own console-script entry point) that exposes every /tools/* route as an MCP tool for an LLM agent host such as Claude Desktop or Claude Code. It talks to the HTTP API purely over httpx -- it never imports apm_connectors.tools or

apm_connectors.graph directly -- so it adds no new enforcement code and needs none: an MCP tool call to gmail_send still ends up as a plain POST /tools/gmail/send, hitting the exact same start_action → interrupt() → approval pipeline as any other caller. A misbehaving or adversarial agent has no more power here than a script hitting the REST API directly.

Eighteen tools in total mirror the eighteen /tools/* routes one-to-one, plus a single shared decide_action tool for the decision route. Tool docstrings here do real work beyond documenting for a human reader — they're the natural-language interface an LLM reads at call-decision time to translate a free-text request into an actual call with the right arguments.

6. Deployment — AWS ECS on Fargate

infra/aws/ecs-fargate/ (Terraform) builds and pushes the image, then stands up an ECS Fargate service behind a public Application Load Balancer that health-checks /health, in the account's default VPC. terraform apply does the whole thing in one command — including the docker build/push itself, via a local-exec provisioner — no separate CI/CD system. Every connector credential and DATABASE_URL are optional Terraform variables, defaulting to empty exactly like an unfilled local .env; secrets are stored as SSM SecureString parameters, never plain environment variables, and resolved by ECS at task startup.

Two things Terraform's own diff genuinely cannot see, both compensated for by a null_resource that hashes the actual content and forces a fresh deployment when it changes: a new image pushed to the same :latest tag (the task definition's image string is unchanged either way), and an SSM secret's value rotating in place (its ARN — all the task definition's secrets block ever references — doesn't change, and ECS only resolves secrets once, at task startup).

The Postgres instance itself is provisioned separately (RDS, or an external managed Postgres such as Neon) and wired in with a single database_url Terraform variable, stored the same SSM-SecureString way as every other secret — Terraform doesn't stand up the database, only points the running task at it.

7. Vocabulary cheat-sheet

Term	Meaning here
`BaseTool / Capability`	The common connector interface (tools/base.py) and its read/write/action capability flags.
`dry_run guard`	require_dry_run_guard — a connector-level defense in depth that logs every real vs. dry-run call, independent of the graph's interrupt() gate.
`interrupt() / Command(resume=...)`	LangGraph's pause/resume primitive — the actual mechanism that makes a write physically wait for a human decision.
`Checkpointer`	Where a paused graph's execution state lives between the pause and the resume — MemorySaver (in-process) or PostgresSaver (durable).
`StateStoreProtocol`	The method surface every state store implementation (file-backed or Postgres) satisfies — what callers actually depend on.
`DATABASE_URL`	The single setting that switches both the state store and the checkpointer from in-memory/local-file to Postgres, together.
`MCP (Model Context Protocol)`	The standard this package's separate apm_connectors_mcp package uses to expose /tools/* as tools for an LLM agent host.

That's the whole system: a thin, reasoning-free HTTP layer over five connectors, gated by one interrupt-based approval mechanism, backed by a persistence layer that can be either zero-infrastructure or durable behind a single setting, reachable by a script, a reasoning layer, or an LLM agent through the same contract either way.